

Mesurer la similarité

Représenter des documents comme des points dans l'espace : sacs de mots,
distance et angle

Traduit et adapté depuis Bootstrap World (Fall 2026)

Licence Creative Commons 4.0 International

*Ces supports ont été développés en partie grâce au soutien de la National Science Foundation
(bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).*

Table des matières

1	Comment les ordinateurs « voient »-ils ?	3
1.1	Lancement	3
1.2	Exploration : pixels et attributs calculés	4
1.3	Synthèse	9
2	Distance dans l'espace image	10
2.1	Lancement : « sont-elles identiques ? »	10
2.2	Exploration : des deux aux trois dimensions	15
2.3	Synthèse	17
3	Mesurer les angles dans l'« espace image »	18
3.1	Lancement : angle de similarité	18
3.2	Exploration : angle-similarity en Pyret	19
3.3	Synthèse	23

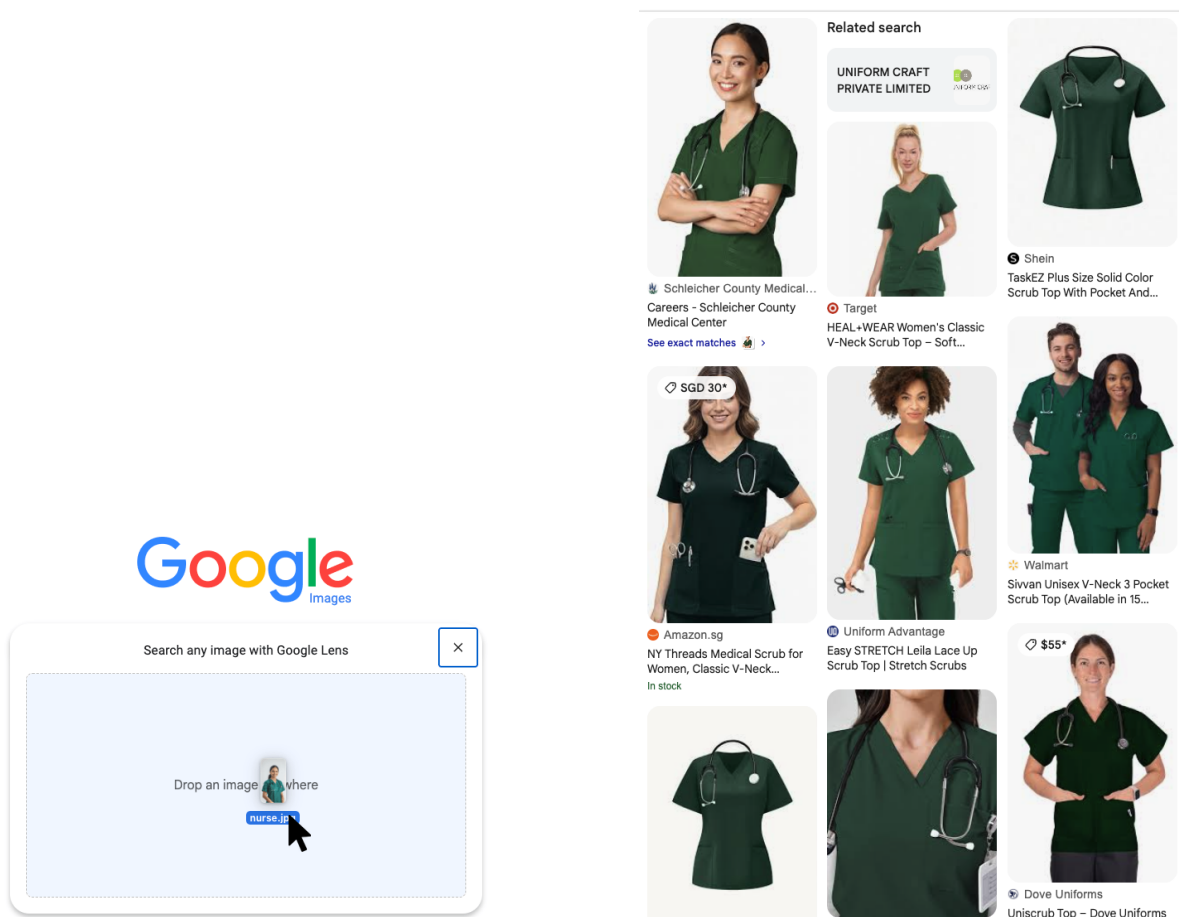
Comment les ordinateurs « voient »-ils ?

Objectif de la section

Les élèves explorent les façons dont les attributs des images peuvent être calculés, en utilisant diverses fonctions.

Lancement

Dans Google Image Search, vous pouvez faire glisser une image dans la barre de recherche et demander à Google de trouver d'autres images *similaires*. La recherche d'images est essentiellement un **système de recommandation**, où l'utilisateur·trice téléverse une image qu'il aime et Google produit une liste de correspondances recommandées.



Gauche : faire glisser une image dans la barre de recherche Google Images. Droite : les premiers résultats d'une recherche d'images similaires.

Activité

- Ouvrez le **fichier Images Similaires** (Images-Similaires-Starter.arr) sur votre ordinateur et enregistrez-en une copie qui n'appartient qu'à vous. Cliquez sur « Run ».
- Il y a beaucoup de définitions dans ce fichier ! Expérimentez avec elles dans la Zone d'Interactions pour voir ce qu'elles font.



FIGURE 1 – Les 9 images de montagnes utilisées dans le fichier Pyret, étiquetées avec leurs numéros d'identification.

Activité

Évaluez `mountains` dans la **Zone d'Interactions**, puis cliquez sur la vignette pour l'agrandir.

- **Quels attributs ces neuf images partagent-elles ? Qu'est-ce qui les rend similaires ?**
- **Quels attributs distinguent ces neuf images ? Qu'est-ce qui les rend différentes ?**
- **Laquelle est la *plus* similaire à `sun1` : `day1` ou `day2` ? Pourquoi ?**
 - `day1` — comme les montagnes dans `sun1`, celles de `day1` n'occupent qu'une étroite bande de la photo. Comme `sun1`, `day1` inclut un plan d'eau au premier plan et un soleil à l'arrière-plan.
 - `day2` — les palettes de couleurs de `sun1` et `day2` sont principalement blanc, marron, et bleu.

Exploration : pixels et attributs calculés

Les ordinateurs ne perçoivent pas le monde comme nous. Là où nous voyons une magnifique scène de montagne, un ordinateur ne voit qu'une grille de **pixels** — des minuscules points qui

composent l'image.

Chaque pixel a une quantité particulière de *Rouge*, *Vert* et *Bleu*. Pour nos besoins, nous utiliserons une échelle de 0 à 255 pour enregistrer la quantité de chaque couleur qui compose un pixel.

Point clé

- Le **blanc** est composé de toutes les couleurs, donc un pixel blanc aurait une valeur de 255 pour le Rouge, le Vert *et* le Bleu : (255, 255, 255).
- Un pixel **noir** est l'*absence* de couleur, donc il aurait des zéros pour les trois : (0, 0, 0).
- Un pixel **rouge** utilise tout le rouge possible et ni bleu ni vert : (255, 0, 0).
- Un pixel **orange** pourrait utiliser la plupart du rouge, un peu de vert, et pas de bleu : (254, 165, 0).

Activité

- Dans la **Zone de Définitions**, trouvez la définition de `red-blue-tile`, `pixel`, et `pixel-dominant-rgb`.
- Évaluez `red-blue-tile`. **Comment décririez-vous ce que vous voyez?**
 - Un petit carré rouge avec un plus petit carré bleu centré dessus.
- Évaluez `pixel` et `pixel-dominant-rgb` pour vous faire une idée des informations que l'ordinateur va utiliser pour calculer ces attributs.
- **Quelle est la similarité entre le résumé initial de l'ordinateur et le nôtre?**



FIGURE 2 – Un papillon monarque en noir et blanc.

Point clé

Des groupes de pixels pour calculer des attributs

Les ordinateurs peuvent regarder des *groupes de pixels* pour calculer des valeurs. Par exemple :

- **Couleur** — « à quel point l'image est-elle bleue ? »
- **Symétrie** — « à quel point les pixels d'un côté de l'image sont-ils similaires aux pixels de l'autre côté ? »
- **Luminance** — mesure la tendance centrale : « quelle est la luminosité moyenne de cette image ? »
- **Entropie** — mesure la dispersion : « quelle est la variation de luminosité ? »

Activité

Évaluez chacune des expressions suivantes :

- `image-symmetry-horizontal(red-sq)`, puis `image-symmetry-vertical(green-t)` et `half-and-half`. Qu'en est-il de `image-symmetry-vertical(half-and-half)` ?
- `image-luminance(black-sq)`, puis `white-sq`. Que pensez-vous que sera la luminance de `half-and-half` ?
- `image-entropy(black-sq)`, puis `white-sq`. Que pensez-vous que sera l'entropie de `half-and-half` ?
- `image-to-color-list(square(5, "solid", "orange"))`
- `dominant-rgb-colors(square(5, "solid", "orange"))`
- **Que remarquez-vous ? Que vous demandez-vous ?**

Activité

- Jetez un œil aux lignes 31–45 de notre Zone de Définitions.
 - Notez que nous définissons une *table* appelée *corpus*.
 - Il y a des colonnes pour l'identifiant (ID), le document image (DOC), si l'image est LIKED ou DISLIKED, et aussi pour les TAGS descriptifs.
- Les lignes 47–52 dirigent l'ordinateur pour construire une table d'attributs calculés en utilisant les fonctions avec lesquelles nous avons expérimenté.

Activité

- Évaluez `computed-with-color-names` dans la Zone d'Interactions.
- Faites défiler vers le bas pour trouver la ligne de `red-blue`.
- Maintenant évaluez `computed`. Quelles colonnes sont différentes ? Quelles sont les mêmes ?
- **Que remarquez-vous ? Que vous demandez-vous ?**

Point clé

Les colonnes de ces tables

- Beaucoup des échelles que vous verrez ici vont de 0 à 255. Nous utilisons une couleur 8 bits pour les formes simples (pensez à la Nintendo d'origine!), ce qui permet 256 valeurs uniques possibles.
- LUMINANCE — la luminosité moyenne d'une image. `image-luminance(black-sq)` vaut zéro, tandis que `image-luminance(white-sq)` vaut 255.
- ENTROPY — mesure le caractère aléatoire de la luminosité entre les pixels. Pour `black-sq` et `white-sq`, l'entropie est nulle (aucun caractère aléatoire). Mais `image-entropy(half-and-half)` vaut 1 — les pixels noirs et blancs ont chacun une chance sur 2.
- SYMMETRY-V et SYMMETRY-H — calculées sur une échelle de 0 à 1, 1 étant une symétrie parfaite et 0 une asymétrie parfaite.
- DOMINANT-RGB-COLORS — liste la couleur dominante (rouge, vert, ou bleu) pour chaque pixel, dans l'ordre. Notre `red-sq` fait 10×10 , donc on s'attend à voir « red » 100 fois. Les pixels de `black-sq`, en revanche, sont *également répartis* entre rouge, vert, et bleu (tous à zéro), donc il n'y a *pas de couleur dominante*. **Cette colonne ne se trouve que dans `computed-with-color-names`.**
- blue, green, red — ces colonnes résument les informations de DOMINANT-RGB-COLORS, en donnant le nombre de fois où chaque couleur a été listée comme dominante. La somme des valeurs de ces trois colonnes devrait être égale au nombre total de pixels. **Ces trois colonnes ne se trouvent que dans `computed`.**

En informatique, c'est une convention d'utiliser l'ordre « Rouge, Vert, Bleu » pour parler des valeurs de pixel, mais notre fonction `add-bag-cols` (ligne 52) met toujours les colonnes dans l'ordre *alphabétique*. Dans Desmos et nos fiches de travail, nous avons gardé la convention RGB. Mais ces conventions différentes pourraient être une source de confusion pour certain-es élèves.

Réflexion

- En quoi **computed** est-elle différente de **computed-with-color-names**?
 - La colonne DOMINANT-RGB-COLORS manque dans **computed**. À la place, elle a été remplacée par trois autres colonnes : **blue**, **green**, et **red**.
- D'où pensez-vous que viennent les 3 dernières colonnes?
 - Elles sont un **compte** du nombre de fois où chaque couleur est apparue dans la chaîne d'origine.
- En comptant le nombre de fois où chaque mot apparaît dans la chaîne, quelle information avons-nous *perdue*?
 - Nous perdons toute information sur l'**ordre**. Nous pourrions prendre les pixels d'une image et les mélanger complètement, et les deux images auraient toujours exactement le même nombre de pixels rouges, verts, et bleus.

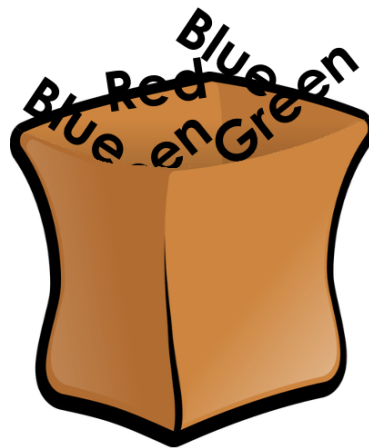


FIGURE 3 – Un sac de dessin animé, rempli avec les mots « Red », « Green » et « Blue ».

Point clé

Sac de mots (*Bag of Words*)

Compter les mots et ignorer leur ordre est une technique pour extraire la *fréquence* des mots dans une chaîne. C'est comme si nous découpons tous les mots d'une chaîne et les mettions dans un sac : nous savons toujours combien il y en a de chaque, mais nous ne savons pas dans quel ordre ils sont venus.

Pour l'instant, nous ne comptons que des couleurs, mais vous pourriez l'imaginer utile pour trouver la similarité entre des essais (en comptant des mots), des stratégies d'échecs (en comptant des coups), etc.

Réflexion

Rappelez-vous nos images de montagnes.

- **Comment pourrions-nous utiliser ces fonctions pour identifier le fait qu'une image enneigée est *similaire* à une autre?**
 - Elles sont toutes les deux à dominante bleue. On pourrait examiner les couleurs de quelque manière.
 - Elles sont toutes les deux les plus uniformes. On pourrait utiliser la colonne entropie?
- **Quelle image a le plus de rouge?** (sunset)
- **Laquelle a le plus de vert?** (celle avec l'herbe)
- **Laquelle pensez-vous aura le plus d'entropie? de luminance? de symétrie?**

Synthèse

Réflexion

- **Qu'est-ce qu'un modèle sac de mots?**
- **Quelle est la différence entre entropie et luminance?**

Point clé

Un modèle est un résumé mathématique d'un jeu de données.

- **Que résume un sac de mots à propos d'une image, dans cet exemple?**
 - Le nombre de pixels à dominante rouge, verte, ou bleue.
- **Qu'est-ce qu'il laisse de côté?**
 - Le reste de la couleur de chaque pixel, l'ordre des pixels, et bien plus!
- **Avons-nous un seul grand modèle ici, ou une table pleine de modèles par image?**
 - **Une table pleine de modèles par image!** Rappelez-vous, un modèle sert à *généraliser*. Mais ces sacs de mots ne s'appliquent qu'aux images d'où ils proviennent — ils ne nous disent rien d'autres images. Notre table a toujours autant de lignes qu'avant! Nous avons résumé à *l'intérieur* de chaque image. Mais il n'y a eu aucun résumé *entre* les images.

Mesurer la distance dans l'espace image

Objectif de la section

Les élèves découvrent la *distance* comme notion de similarité. Cela leur demande de visualiser les images dans un système spatial — d'abord en une dimension, puis en deux, puis éventuellement en trois.

Lancement : « sont-elles identiques ? »

Une façon de comparer des images est simplement de demander : « sont-elles identiques ? » C'est du tout ou rien : soit les images correspondent *exactement*, soit non.

Réflexion

- **Quand nous cherchons des recommandations, espérons-nous une liste de choses qui sont exactement les mêmes ?**
 - Pas habituellement ! Nous voulons quelque chose de plus nuancé qui recommande des choses *similaires*.



FIGURE 4 – Les 9 images de montagnes avec leurs identifiants.

Réflexion

- **Si nous cherchions une image similaire à une autre avec beaucoup d'herbe, une option pourrait être de se concentrer sur les images avec une palette de couleurs similaire. Quelle colonne de couleur s'attendrait-on à avoir des valeurs similaires pour deux images avec beaucoup d'herbe ?**
 - green (vert).

Activité

- Voyons comment les images se comparent sur l'échelle "green" en évaluant `image-dot-plot(computed, "green", thumbnail)`.
- Trouvez grass et grass-sun sur le graphique.
- **Que remarquez-vous?**

Point clé**La percée conceptuelle**

Ces colonnes peuvent chacune être représentées comme une *droite numérique*, avec chaque image située *quelque part sur cette ligne*. Cela nous permet d'imaginer nos images existant comme des points dans l'espace, et ouvre de nouvelles façons de mesurer la similarité en pensant à la distance entre ces points.

Nous avons utilisé la colonne "green" pour définir une droite numérique — un « Espace Vert » — où chaque image était tracée selon le nombre de pixels à dominante verte qu'elle avait. Et nos deux images herbeuses sont proches l'une de l'autre.

- **Comment pourrions-nous quantifier la similarité entre les images dans cet « Espace Vert »?**
 - Nous pourrions calculer la distance entre elles sur cette droite numérique.



FIGURE 5 – Un plan avec deux points $(2, 2)$ et $(6, 1)$. Le triangle rectangle avec son hypoténuse entre ces points est dessiné en gris.

Point clé

La parabole du marieur

Une mère va voir le marieur du village. « Voici mon enfant, » dit-elle, « peux-tu lui trouver un bon match ? » Le marieur réfléchit à tous ceux qu'il connaît, et note le nom du partenaire le plus compatible, du second plus compatible, et ainsi de suite.

Cette histoire résume assez bien ce que nous essayons de faire.

- L'ordinateur est notre marieur.
- Nous apportons une image spécifique et lui demandons de trouver une correspondance.
- Nous avons des qualités spécifiques qui nous tiennent à cœur.
- Le « marieur » pense à toutes les images du corpus, et les classe par ordre décroissant de compatibilité.

```
# distance-similarity :: (Table t, String id,  
List<String> dimensions) -> Table
```

Code Pyret

Par exemple, nous pourrions demander à notre fonction de marieur une liste de correspondances pour notre image "grass", en nous souciant uniquement de la quantité de green :

```
distance-similarity(computed, "grass", [list: "green"])
```

Activité

- Retournez au fichier **Images Similaires** et trouvez les lignes 26–40 de la Zone de Définitions.
 - Pour l'instant, aucune image n'est aimée, non aimée, ou étiquetée. En définissant la table ici dans le code, nous nous sommes préparés à pouvoir faire des changements.
- Dans la colonne TAGS :
 - changez "" en "grass" pour les images avec de grandes quantités d'herbe.
 - changez "" en "snow" pour les images enneigées.
 - changez "" en "sun" pour les quatre images avec un soleil visible.
 - ajoutez des étiquettes pour *water* et *red* (pour donner une deuxième étiquette à une image, utilisez une virgule, par exemple "grass, sun").
- Choisissez les **deux images de montagnes que vous préférez**, et changez les valeurs de ces images dans la colonne LIKED de *false* à *true*.
- Cliquez sur « Run » pour charger vos changements.
- Utilisez-les pour compléter la première section de *Tracer des images*.

Réflexion

- Combien d'images avez-vous étiquetées avec **"grass"** ? 2.
- **distance-similarity(computed, "grass", [list: "green"])** les a-t-elle toutes triées en haut de la table ?
 - Oui. `distance-similarity` renvoie toujours une table avec les lignes triées de la plus à la moins similaire.
- Quelle est la **distance-similarity** pour la première ligne ? Pourquoi ?
 - 0, parce que notre ligne **"grass"** est identique à elle-même.
- Quelle caractéristique a conduit Pyret à calculer **day1** et **day2** comme similaires aux images herbeuses dans notre Espace Vert ?
 - Le nombre de pixels à dominante verte dans les images ! L'œil humain reconnaît ces pixels à dominante verte comme des étendues de forêt.

Réflexion

- Comparer les images en fonction du nombre de pixels verts est un début, mais c'est une compréhension assez limitée. Nous devons pouvoir comparer les images sur plusieurs colonnes de données !
- Comment pourrions-nous tracer des images en utilisant deux colonnes de données ?
 - Sur un nuage de points.
- Pouvons-nous mesurer la distance entre 2 points en 2 dimensions ?
 - Oui ! C'est ce que fait la formule de distance.

Activité

Complétez la deuxième section de *Tracer des images* pour explorer nos images dans l'« Espace Bleu-Vert ».

Point clé

En choisissant *deux* colonnes de notre jeu de données et en les utilisant comme axes, vous avez défini un « Espace Bleu-Vert » à 2 dimensions ! Chacune des images de notre corpus existe maintenant comme un point dans cet espace.

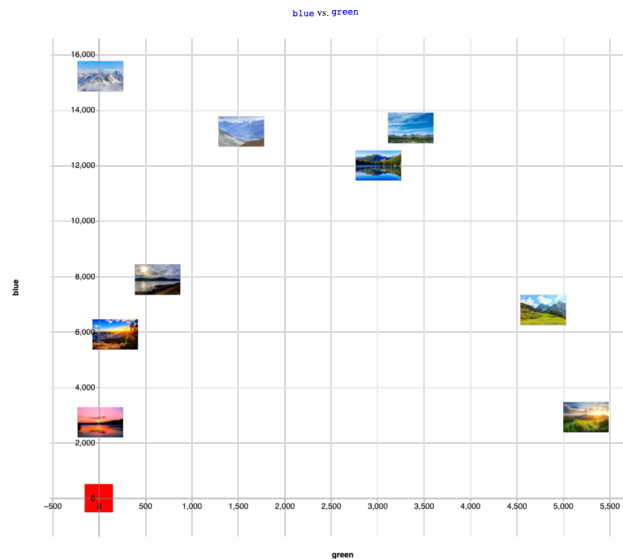


FIGURE 6 – Un nuage de points 2D montrant les images tracées sur les axes bleu vs. vert.

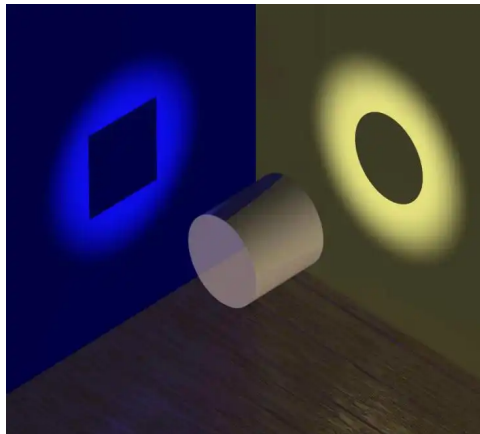
Réflexion

- **Quels groupes d’images avez-vous trouvés dans notre « Espace Bleu-Vert » ?**
 - images avec du rouge – en bas à gauche.
 - neige – en haut à gauche.
 - herbe – à droite.
- **Comment l’« Espace Bleu-Vert » s’est-il débrouillé pour rassembler les images avec un soleil visible ?**
 - Elles sont un peu partout !
- **Quelle distance est calculée en 2 dimensions ?**
 - Quand les images ne partagent pas de coordonnée, distance-similarity calcule la distance *en diagonale*.
 - Nous pouvons calculer la distance entre deux points en utilisant la formule de distance.
- **Pourrions-nous trouver la distance entre 2 points en 3 dimensions ? Pourquoi ou pourquoi pas ?**
 - Oui !
 - Vous n’avez peut-être pas encore appris à calculer la distance entre 2 paires de coordonnées dans l’espace 3D, mais nous pouvons certainement mesurer la distance entre deux objets dans notre classe avec une règle !

Activité

Complétez la dernière section de *Tracer des images*.

Exploration : des deux aux trois dimensions



Rappelez-vous cette image d'un objet 3D projeté sur des plans 2D. L'ombre carrée et l'ombre circulaire nous montrent chacune une partie de l'histoire, mais la vraie forme est 3D.

Point clé

Il en va de même pour notre jeu de données ! Nos images sont décrites par 3 couleurs différentes (Rouge, Bleu, et Vert).

Comme les ombres dans cette image, nos deux nuages de points dans l'« Espace Bleu-vs-Vert » et l'« Espace Bleu-vs-Rouge » ne sont que des *projections* de points 3D sur les plans Z-X et Z-Y.

Notre « espace des couleurs » est tridimensionnel. Les nuages de points ne montrent les données que d'un seul point de vue.

Activité

- Ouvrez le nuage de points RVB (Desmos).
- La table en haut du panneau contient les valeurs Rouge, Vert, et Bleu de notre table `computed`.
- Trouvez le point de l'image `grass`, qui est tracé comme un point violet.
- Le nuage de points pour Bleu-vs-Vert est visible contre la « paroi arrière » du cube, chaque image étant un point **vert**. Comparez-le au nuage de points Pyret sur *Tracer des images*.
- Complétez les deux premières sections de *De deux à trois dimensions*.

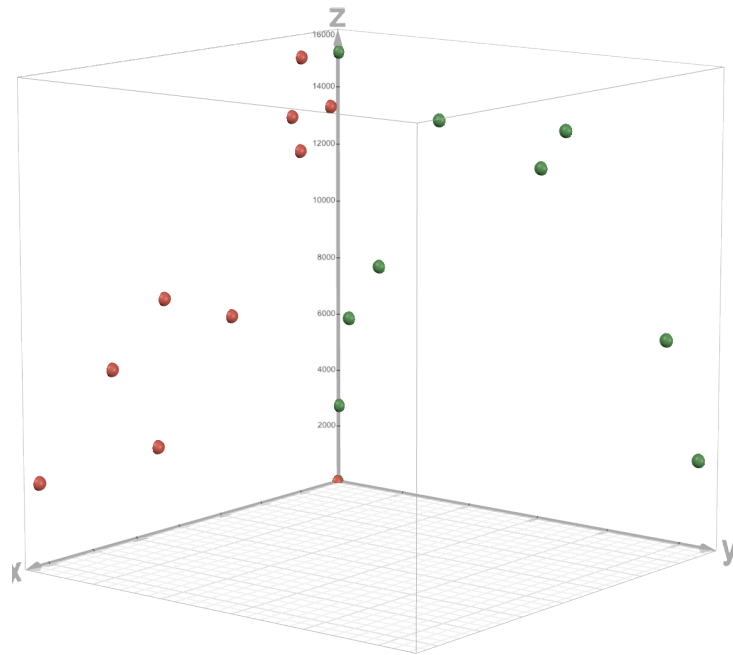


FIGURE 7 – Un nuage de points 3D montrant les images sur les axes bleu-vs-rouge-vert, avec les projections de ces points sur les plans bleu-vs-rouge et bleu-vs-vert.

Point clé

Nous ne pouvons pas *voir* plus de 3 dimensions à la fois avec nos yeux. Mais l'ordinateur peut calculer la distance dans de nombreuses dimensions. **Les maths restent vraies**, et nous pouvons utiliser *distance-similarity*, avec autant d'attributs que nous le voulons.

Activité

Comment **distance-similarity** marche-t-elle avec 3 dimensions pour les images avec de l'herbe ?

```
distance-similarity(computed, "grass", [list: "red", "green", "blue"])
```

- Essayez-la avec l'ID "snow2", pour voir comment elle marche avec les images de **neige**.
- Essayez-la avec l'ID "red2", pour voir comment elle marche avec les images de **carrés rouges**.
- Essayez-la avec l'ID "grass-sun" pour voir comment elle marche avec les images de **soleil**.

Essayez d'ajouter d'autres colonnes à votre liste de dimensions. Pouvez-vous identifier un espace de dimension *supérieure* qui marche bien pour les quatre groupes ?

Point clé**Limites de *distance-similarity***

Dans l'ensemble, *distance-similarity* est une amélioration énorme par rapport à la question « sont-elles identiques? ». Mais comme *marieur*, elle souffre encore de deux problèmes majeurs :

- **Elle se concentre sur le *compte* plutôt que sur la *proportion*** : par exemple, une grande photo de montagne a beaucoup plus de pixels rouges qu'un petit carré rouge – et apparaît donc « plus proche » du grand carré rouge que du petit – même si seule une petite fraction de la photo de montagne est rouge.
- **Elle ignore l'*orientation*** : les humains voient deux images « majoritairement rouges » comme similaires, mais la formule de distance ne le fait pas. Elle ne calcule que l'espace entre les points, ignorant où ils se situent *réellement* sur le graphique.

Synthèse

Réflexion

- **Quel rôle chaque colonne joue-t-elle dans la définition de l'espace dans lequel les images sont comparées?**
 - Chaque colonne est associée à un axe.
- **Les nuages de points sont 2D et nos données ont beaucoup plus de dimensions. Les nuages de points nous disent-ils vraiment quelque chose sur nos données?**
 - Oui! Tout comme nous ne pouvons pas voir une sculpture entière depuis un seul point de vue et que n'importe quel angle nous en dit beaucoup sur la sculpture, les nuages de points manquent d'informations sur d'autres colonnes mais peuvent nous dire beaucoup sur les relations des données dans le plan que nous regardons.

Mesurer les angles dans l'« espace image »

Objectif de la section

Les élèves découvrent la *mesure d'angle* comme notion de similarité. Cela renforce la visualisation des images dans un système spatial.

Lancement : angle de similarité

Essayons une autre forme de similarité, qui repose sur l'idée que les images « à dominante rouge » se regrouperont toutes sur un axe, et les images « à dominante bleue » sur l'autre. La **similarité d'angle** compare les angles formés par deux rayons tracés depuis deux points jusqu'à l'origine.

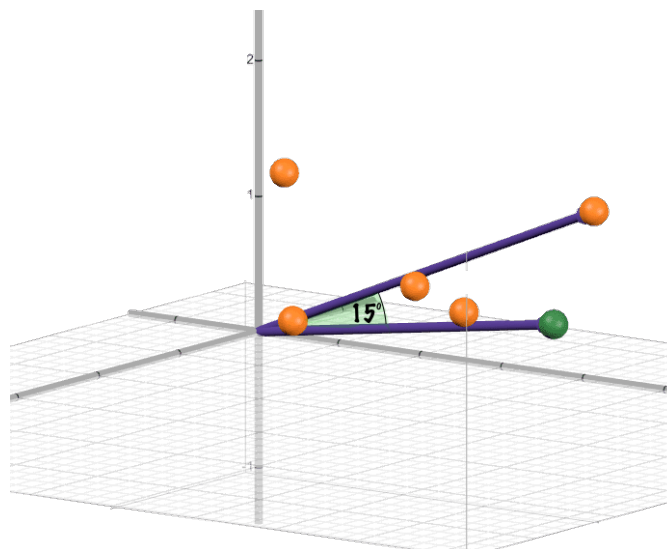


FIGURE 8 – Un système de coordonnées 3D avec des points orange représentant le prix vs. la qualité des frites vs. la qualité des burgers de divers restaurants. Un point vert représente In-N-Out, et des flèches pointent vers lui et le point orange le plus proche. Des lignes sont tracées de ces deux points à l'origine, avec l'angle entre les lignes mesuré à 18 degrés.

Activité

Complétez la dernière section de *De deux à trois dimensions*.

Point clé

Nous pouvons tracer des lignes (appelées « rayons ») depuis l'origine jusqu'à n'importe quel de nos points-image dans l'espace. Au lieu de calculer la distance entre ces points, nous pouvons calculer *l'angle entre ces lignes*.

- Deux points identiques auront un angle de 0° , signifiant « la plus grande similarité possible ».
- Deux points qui tombent sur des axes totalement différents (tout-bleu vs. tout-rouge, par exemple) auront un angle de 90° , signifiant « la moindre similarité possible ».

Cela résout les deux problèmes de *distance-similarity* :

- **Compte vs. proportion** : l'angle entre nos deux carrés rouges sera de zéro, puisqu'ils tombent tous deux exactement sur l'axe red. Cela identifie correctement que nos deux carrés rouges sont similaires.
- **Orientation** : la mesure d'angle, par définition, décrit à quel point deux rayons sont orientés de façon proche.

Réflexion

- **La similarité d'angle fonctionne-t-elle en 1 dimension ? Aurions-nous pu l'utiliser dans notre « Espace Vert » ?**
 - Non ! Une limite de la similarité d'angle est que l'angle sera **toujours de zéro** à moins que nous comparions des documents sur plusieurs dimensions.
- **Les images sont-elles les seules choses que nous pouvons tracer dans un espace à 3 dimensions et utiliser la similarité d'angle ?**
 - Non ! Nous pourrions tracer n'importe quelles données avec des attributs quantitatifs.

Activité

Utilisons notre connaissance des angles pour estimer la similarité d'angle de quelques points dans un espace colorimétrique 2D.
Complétez *Similarité d'angle* (débranché).

Exploration : angle-similarity en Pyret

Bien sûr, Pyret possède aussi une fonction « marieur » pour la similarité d'angle !

```
# angle-similarity :: (Table t, String id, List<String>
dimensions) -> Table
```

Point clé

Ce marieur fait exactement le même travail que `distance-similarity` :

- Nous apportons une image spécifique et lui demandons de trouver une correspondance.
- Nous avons des qualités spécifiques qui nous tiennent à cœur.
- Le « marieur » pense à toutes les images du corpus, et les classe par ordre décroissant de compatibilité. Mais cette fois, il utilise **l'angle entre les rayons** plutôt que la distance entre les points.

Activité

Complétez *Similarité d'angle* (Pyret).

La mesure des angles est-elle plus proche de la perception humaine que la mesure de distance?

Comment `angle-similarity` marche-t-elle dans notre espace colorimétrique 3D pour les images avec de l'herbe?

```
angle-similarity(computed, "grass", [list: "red", "green", "blue"])
```

- Essayez-la avec l'ID "snow2", pour voir comment elle marche avec les images de **neige**.
- Essayez-la avec l'ID "red2", pour voir comment elle marche avec les images de **carrés rouges**.
- Essayez-la avec l'ID "grass-sun" pour voir comment elle marche avec les images de **soleil**.
- Explorez s'il est utile d'ajouter nos colonnes `WIDTH` et `HEIGHT`.

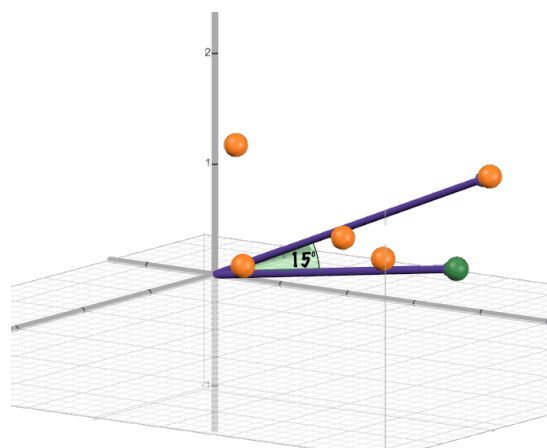


FIGURE 9 – Le même système de coordonnées 3D avec les rayons et l'angle entre les points mesuré.

Point clé

La similarité d'angle nous a vraiment aidés à améliorer la « perception » de la machine sur ce qui rend deux images similaires. À partir d'une image, nous sommes capables de produire de bien meilleurs résultats de recherche ! En repensant à la notion de recherche comme un **système de recommandation**, vous pourriez aussi imaginer faire la même chose avec d'autres types de données : par exemple choisir une chanson et demander des recommandations similaires !

Notes pour l'enseignant·e**Ce n'est pas juste des maths du collège ?**

Si ! Les mêmes points et angles que vos élèves tracent et calculent au collège sont au cœur des calculs d'apprentissage automatique qui alimentent tant de choses dans le monde d'aujourd'hui. Même les détecteurs de plagiat qui vérifient peut-être leurs essais calculent des angles. Donc si vos élèves demandent « quand allons-nous jamais utiliser ça ? », vous pouvez leur dire « vous le faites déjà, tout le temps ».

Si vous avez des élèves plus âgé·es qui ont appris de la trigonométrie, vous voudrez peut-être partager que les vrais systèmes d'apprentissage automatique utilisent le *cosinus* de l'angle. Il y a deux raisons à cela :

- C'est plus simple de calculer le cosinus directement. (En fait, dans Pyret, `angle-similarity` calcule d'abord le cosinus, puis convertit le résultat en angle !)
- L'angle lui-même est une valeur quelque peu gênante. L'idée que 0° signifie « le plus similaire » et que 90° (l'angle le plus grand) signifie le *moins* de similarité peut être contre-intuitive. En revanche, le cosinus a une plage numérique agréable pour les angles entre 0° et 90° , allant de 1 à 0, ce qui le rend pratique dans d'autres contextes mathématiques.

Pour les besoins de ce programme, vous pouvez ignorer la différence entre similarité d'angle et de cosinus ! Mais pour les élèves qui connaissent le cosinus et sont curieux·ses d'explorer, tous les fichiers de démarrage de notre parcours IA ont accès à une fonction `cosine-similarity`.

Point clé**Limitations par rapport à des outils comme Google Image Search**

Notre système a encore certaines lacunes comparé à des outils comme Google Image Search :

- Nous ne pouvons *rechercher que par document* : nous choisissons une image et trouvons des images similaires.
 - Les moteurs de recherche d'images nous permettent de taper un mot comme « herbe » !
 - Les applications de streaming musical comme Tidal nous permettent de taper un mot comme « jazz », plutôt que de nous demander une chanson spécifique.
- Nous ne recommandons des correspondances que sur la base d'un seul document : nous ne pouvons pas trouver des images, des chansons, ou autre chose sur la base d'une *collection* de documents aimés ou non aimés.



FIGURE 10 – Un robot tenant un livre avec un pouce levé sur la couverture.

Certains systèmes d'IA sont capables de reconnaître des parties d'une image (« personne », « voiture », etc.). Cela relève d'une classe d'algorithmes d'IA appelée « Vision par Ordinateur », qui s'appuie fortement sur des concepts plus avancés — et plus intensifs en calcul — que nous ne couvrirons pas ici.

Nous allons utiliser nos cerveaux humains pour voir ces éléments dans nos images, puis explorer si un algorithme piloté par les données peut nous aider à recréer les technologies de recherche d'images, même sans la technologie de Vision par Ordinateur.

Synthèse

Réflexion

- Il est évident pourquoi **angle-similarity** fonctionne en deux dimensions : vous pouvez utiliser un rapporteur sur une feuille de papier au besoin ! Comment pourriez-vous expliquer qu'elle fonctionne encore en trois dimensions ?
 - Tout ce dont nous avons besoin, c'est de deux rayons partant de l'origine, formant un « V » comme les brins d'une fronde. Peu importe comment vous tournez et tordez le « V », il définira toujours un plan sur lequel nous pouvons utiliser un rapporteur.
- Étant donné ce corpus d'images, qui inclut nos carrés rouges, pourquoi ne pouvons-nous pas utiliser la similarité d'angle sur un Espace Bleu-Vert ?
 - La similarité d'angle exige de tracer deux rayons pour que l'angle entre eux puisse être mesuré. Mais nos carrés rouges ont zéro comme valeur pour `blue` et `green`, donc leurs coordonnées dans l'Espace Bleu-Vert seraient $(0, 0)$. Il est impossible de tracer une ligne entre un point et lui-même, donc il n'y a aucun moyen de mesurer l'angle entre nos carrés rouges et toute autre image.
- Y a-t-il des limites à l'utilisation des mesures d'angle pour la similarité, où la distance pourrait être meilleure ?
 - Si nous ne mesurons qu'en 1 dimension, l'angle sera toujours de zéro.

Annexe A : Trois mesures de similarité

Mesure	Fonction Pyret	Trade-off
Concordance exacte	« sont-elles identiques ? »	Tout ou rien. Trop rigide pour des recommandations.
Distance euclidienne	<code>distance-similarity</code>	Compte la distance entre les points. Ignore la proportion (grande vs. petite image) et l'orientation.
Similarité d'angle	<code>angle-similarity</code>	Mesure l'angle entre les rayons depuis l'origine. Capture la proportion et l'orientation, mais nécessite au moins 2 dimensions.

Annexe B : Fichiers du projet

Fichier	Description
<code>Images-Similaires-Starter.arr</code>	Fichier de démarrage Pyret : charge <code>ai-library.arr</code> depuis Bootstrap, fournit <code>distance-similarity</code> , <code>angle-similarity</code> , <code>decorate-image-table</code> , <code>add-bag-cols</code> , etc.
<code>mesurer-similarite.tex</code> <code>images/</code>	Document de cours (ce fichier) 12 images : captures Google Images, montagnes, papillon, sac de mots, nuages 2D/3D, cylindre, robot

À noter : les noms de colonnes (`ID`, `DOC`, `LIKED`, `DISLIKED`, `TAGS`, `LUMINANCE`, `ENTROPY`, `SYMMETRY-V`, ..., `DOMINANT-RGB-COLORS`, `red`, `green`, `blue`) sont codés en dur dans la bibliothèque `ai-library.arr` de Bootstrap (1905 lignes) et ne peuvent pas être traduits sans modifier la bibliothèque elle-même. Tous les commentaires du fichier de démarrage sont en français.

À propos de ces supports

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).

Bootstrap par la Communauté Bootstrap est sous licence Creative Commons 4.0 Unported. Cette licence n'accorde pas la permission d'organiser des formations ou du développement professionnel.

Traduction et adaptation française réalisées en juillet 2026.

Source originale : <https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/measuring-similarity/>